L8 - Datagram protocols and multithreaded servers

Datagram Protocols

Unlike stream protocols such as TCP, in datagram protocols we transmit packets of data, that is, *datagrams*. As with stream protocols, we will study them in the context of both network (UDP) and local sockets (UNIX).

UNIX

A local datagram socket is created similarly to a stream socket, simply by specifying `SOCK_DGRAM` as the protocol type:

```
socket(PF_UNIX, SOCK_DGRAM, 0)
```

All rules related to local sockets apply (`man 7 unix`). It is worth noting that, unlike network datagram sockets (UDP), local ones are reliable - `as on most UNIX implementations, UNIX domain datagram sockets are always reliable and don't reorder datagrams` . This makes them a fairly convenient way to send messages between processes. For example, we do not have the message coalescing problem known from stream protocols.

UDP

A network datagram socket, i.e. a UDP socket, is created analogously to TCP - it is enough to

change the type to `SOCK_DGRAM`, for example:

```
socket(AF_INET, SOCK_DGRAM, 0)
```

for an IPv4 socket (`AF_INET6` for IPv6).

To learn more about UDP, be sure to read `man 7 udp` and the lecture on network sockets. In particular, it is important to understand that UDP is unreliable - messages may be lost, arrive out of order, arrive duplicated, etc. However, because UDP does not have the reliability overhead of TCP, it allows messages to be transmitted with lower latency. It is therefore used in applications where fast data delivery is more important than reliability, such as online multiplayer games. Because UDP is connectionless, it supports so-called *broadcast*, i.e. sending one message to many addresses.

Communication

Datagram protocols are connectionless - we do not call `connect` on the client side, nor `listen` and `accept` on the server side. Instead, we simply have two functions for sending and receiving data:

```
ssize_t recvfrom(int socket, void *restrict buffer, size_t length,
    int flags, struct sockaddr *restrict address, socklen_t *restrict address_le

ssize_t sendto(int socket, const void *message, size_t length,
    int flags, const struct sockaddr *dest_addr, socklen_t dest_len);
```

Read their manual pages - `man 3p recvfrom` and `man 3p sendto`. As you can see, these functions are analogous to `recv` and `send`, but they take two additional parameters: the address and its size.

When receiving a message with `recvfrom`, it is worth paying attention to its specific behavior. This function returns the number of bytes read, but it always reads only one message (datagram) at a time. The `length` parameter indicates the size of the provided `buffer` and means the *maximum* size of a message we expect.

If the message is shorter, it will simply be read in full; if it is longer, the extra bytes will be **ignored and discarded**. Therefore, it is important that `buffer` has an appropriate size, and that the `length` parameter is set to the size of the largest message our program supports, not the size of the message we currently expect. In UDP, we have no guarantee that messages will arrive in the correct order.

In `man 2 recvfrom` you can find some additional information specific to Linux. For example, there is the `recvmsg` function, which in some cases allows better performance and can, for instance, return a flag indicating that a message was truncated. Remember, however, that the contents of that manual page are not part of the POSIX standard and are therefore not portable to other Unix systems (e.g. macOS, BSD). During the laboratory, we generally use only functions from the standard.

The `sendto` function is even simpler to use - it either sends the entire datagram, or it fails and sends nothing.

Task

Goal:

Write client and a server program that communicate over UDP socket. Client task is to send a file divided into proper size datagrams to the server. Server prints out the received data without information about the source. Each packet send to server must be confirmed with return message. If confirmation is missing (wait 0,5 sec.) resend the packet again. If 5 tries in a row fail, client program exits with an error. Both data packets and confirmations can be lost, program must resolve this issue. Server can not print the same part of the file more than once. All metadata (everything apart from file content) send over the udp socket must be converted to int32_t type. You can assume that maximum allowed datagram (all data and metadata) size is 576B. Server can handle 5 concurrent transmissions at a time. If sixth client tries to send data it should be ignored. Server program takes port as its sole parameter, client takes address and port of the server as well as file name as its parameters.

What you need to know:

```
man 7 udp
man 3p sendto
man 3p recvfrom
man 3p recv
```

```
man 3p send
```

Solution `l8-1_server.c`:

```
#include "l8_common.h"

#define BACKLOG 3
#define MAXBUF 576
#define MAXADDR 5

struct connections
{
    int free;
    int32_t chunkNo;
    struct sockaddr_in addr;
};

int make_socket(int domain, int type)
{
    int sock;
    sock = socket(domain, type, 0);
    if (sock < 0)
        ERR("socket");
    return sock;
}

int bind_inet_socket(uint16_t port, int type)
{
    struct sockaddr_in addr;
    int socketfd, t = 1;
    socketfd = make_socket(PF_INET, type);
    memset(&addr, 0, sizeof(struct sockaddr_in));
    addr.sin_family = AF_INET;
    addr.sin_port = htons(port);
    addr.sin_addr.s_addr = htonl(INADDR_ANY);
    if (setsockopt(socketfd, SOL_SOCKET, SO_REUSEADDR, &t, sizeof(t)))
```

```
        ERR("setsockopt");
    if (bind(socketfd, (struct sockaddr *)&addr, sizeof(addr)) < 0)
        ERR("bind");
    if (SOCK_STREAM == type)
        if (listen(socketfd, BACKLOG) < 0)
            ERR("listen");
    return socketfd;
}

int findIndex(struct sockaddr_in addr, struct connections con[MAXADDR])
{
    int i, empty = -1, pos = -1;
    for (i = 0; i < MAXADDR; i++)
    {
        if (con[i].free)
            empty = i;
        else if (0 == memcmp(&addr, &(con[i].addr), sizeof(struct sockaddr_in)))
        {
            pos = i;
            break;
        }
    }
    if (-1 == pos && empty != -1)
    {
        con[empty].free = 0;
        con[empty].chunkNo = 0;
        con[empty].addr = addr;
        pos = empty;
    }
    return pos;
}

void doServer(int fd)
{
    struct sockaddr_in addr;
    struct connections con[MAXADDR];
    char buf[MAXBUF + 1];
```

```
for (int i = 0; i < MAXADDR; i++)
    con[i].free = 1;

while (1)
{
    socklen_t size = sizeof(addr);
    int receivedBytes;
    if ((receivedBytes = TEMP_FAILURE_RETRY(recvfrom(fd, buf, MAXBUF, 0, (st
        ERR("read:");
    buf[receivedBytes] = 0;
    int index = -1;

    if ((index = findIndex(addr, con)) >= 0)
    {
        int32_t chunkNo = ntohl(*((int32_t *)buf));
        if (chunkNo > con[index].chunkNo + 1)
        {
            continue;
        }
        else if (chunkNo == con[index].chunkNo + 1)
        {
            if (ntohl(*(((int32_t *)buf) + 1)))  // last message bit is set
            {
                printf("Last Part %d\n%s\n", chunkNo, buf + 2 * sizeof(int32
                con[index].free = 1;
            }
            else
            {
                printf("Part %d\n%s\n", chunkNo, buf + 2 * sizeof(int32_t));
            }
            con[index].chunkNo++;
        }

        if (TEMP_FAILURE_RETRY(sendto(fd, buf, MAXBUF, 0, (struct sockaddr *
        {
            if (EPIPE == errno)
                con[index].free = 1;
```

```
                else
                    ERR("send:");
            }
        }
    }
}

void usage(char *name) { fprintf(stderr, "USAGE: %s port\n", name); }

int main(int argc, char **argv)
{
    int fd;
    if (argc != 2)
    {
        usage(argv[0]);
        return EXIT_FAILURE;
    }
    if (sethandler(SIG_IGN, SIGPIPE))
        ERR("Seting SIGPIPE:");
    fd = bind_inet_socket(atoi(argv[1]), SOCK_DGRAM);
    doServer(fd);
    if (TEMP_FAILURE_RETRY(close(fd)) < 0)
        ERR("close");
    fprintf(stderr, "Server has terminated.\n");
    return EXIT_SUCCESS;
}
```

Solution `l8-1_client.c`:

```
#include "l8_common.h"

#define MAXBUF 576
volatile sig_atomic_t last_signal = 0;

void sigalrm_handler(int sig) { last_signal = sig; }
```

```
int make_socket()
{
    int sock;
    sock = socket(PF_INET, SOCK_DGRAM, 0);
    if (sock < 0)
        ERR("socket");
    return sock;
}

void usage(char *name) { fprintf(stderr, "USAGE: %s domain port file \n", name);

void sendAndConfirm(int fd, struct sockaddr_in addr, char *sendbuf, char *recvbu
{
    struct itimerval ts;
    if (TEMP_FAILURE_RETRY(sendto(fd, sendbuf, size, 0, (struct sockaddr *)&addr
        ERR("sendto:");
    memset(&ts, 0, sizeof(struct itimerval));
    ts.it_value.tv_usec = 500000;
    setitimer(ITIMER_REAL, &ts, NULL);
    last_signal = 0;
    while (recv(fd, recvbuf, size, 0) < 0)
    {
        if (EINTR != errno)
            ERR("recv:");
        if (SIGALRM == last_signal)
            break;
    }
}

void doClient(int fd, struct sockaddr_in addr, int file)
{
    char sendbuf[MAXBUF];
    char recvbuf[MAXBUF];
    int offset = 2 * sizeof(int32_t);
    int32_t chunkNo = 0;
    ssize_t size;
```

```
    int counter;
    do
    {
        memset(sendbuf, 0, MAXBUF);
        memset(recvbuf, 0, MAXBUF);

        if ((size = bulk_read(file, sendbuf + offset, MAXBUF - offset)) < 0)
            ERR("read from file:");
        *((int32_t *)sendbuf) = htonl(++chunkNo);
        if (size < MAXBUF - offset)
        {
            memset(sendbuf + offset + size, 0, MAXBUF - offset - size);
            *(((int32_t *)sendbuf) + 1) = htonl(1);
        }
        counter = 0;

        do
        {
            counter++;
            sendAndConfirm(fd, addr, sendbuf, recvbuf, MAXBUF);
        } while (*((int32_t *)recvbuf) != (int32_t)htonl(chunkNo) && counter <=

        if (*((int32_t *)recvbuf) != (int32_t)htonl(chunkNo) && counter > 5)
            break;

    } while (size == MAXBUF - offset);
}

int main(int argc, char **argv)
{
    int fd, file;
    struct sockaddr_in addr;
    if (argc != 4)
    {
        usage(argv[0]);
        return EXIT_FAILURE;
    }
```

```
    if (sethandler(SIG_IGN, SIGPIPE))
        ERR("Seting SIGPIPE:");
    if (sethandler(sigalrm_handler, SIGALRM))
        ERR("Seting SIGALRM:");
    if ((file = TEMP_FAILURE_RETRY(open(argv[3], O_RDONLY))) < 0)
        ERR("open:");
    fd = make_socket();
    addr = make_address(argv[1], argv[2]);
    doClient(fd, addr, file);
    if (TEMP_FAILURE_RETRY(close(fd)) < 0)
        ERR("close");
    if (TEMP_FAILURE_RETRY(close(file)) < 0)
        ERR("close");
    return EXIT_SUCCESS;
}
```

There is no connection in UDP protocol, sockets send datagrams “ad hoc”. There is no listening socket. Losses, duplicates and reordering of datagrams are possible!

In this example you will find useful library candidate functions like: make_socket, bind_inet_socket as they have conflicting names with previously recommended functions, you have to rename them.

In this example connection context is more demanding. What constitutes the context here?

► Answer:
The number of packets send of specified file up to given moment is the context here - in other words struct connections.

What data is sent in datagram? What is the purpose of the metadata?

► Answer:
The datagram consists of (1. 32 bit file part number, (2) 32 bit information if this is the last part of the file, (3) the file part. Metadata helps to maintain the context, keep the track of the file being sent (1. and end the transmission (2).

Why and on what descriptors bulk_read and bulk_write are used? Should we extend this use on all the descriptors in the program?

► Answer:
Mentioned functions are used to restart read and write after interruption on IO. Notice that it is restarting both interruption types: before IO starts (EINTR) and the interruption in the middle of IO. Those functions are only used on files as datagrams are sent in atomic way (transfer can not be interrupted). In this program signals are handled in code parts that do not operate on files, still due to code portability bulk_ functions are used.

Do we expect broken connection in this program? Should we add checks in the code?

► Answer:
We do not have connection to break in UDP.

How findIndex works in server code: How addresses are compared? What byte order they are in? What will the function do if the address in new?

► Answer:
Addresses are compared in binary format without conversion to host byte order. We do not need to convert the byte order as we only compare the addresses, we do not display them. If new address is passed to this function it starts a new record for this address in the array (provide there is a space for it).

How this program deals with the duplicates of datagrams?

► Answer:
It keeps an array of active connections “struct connections” with the number of last transferred part. Duplicated parts are not processed.

How this program deals with the reordered datagrams, i.e. when program receives a part that is farther in the file than the next expected?

► Answer:
This can not happen in this program, client will not send such a part without having all the previous parts confirmed by the server.

How this program handles lost datagrams?

► Answer:
Client side re-transmission.

What will happen if the packet with confirmation from the server to the client gets lost?

► Answer:
The client will assume that the last part sent did not get to the server, it will send it again. Server will get the duplicate of the part that is already stored, it will not process it but it will send another confirmation.

What is in the confirmation?

► Answer:
Server sends back exactly the same data as it received.

How timeout on the server response is implemented?

► Answer:
In function sendAndConfirm 0.5s alarm is set (with setitimer) then the program waits for the confirmation on regular not restarted recv call. If signal is received, the recv gets interrupted and the code checks if the timeout triggered.

Why the program converts byte order only the part number and the last part marker and not the rest?

► Answer:
Only those two are send in binary integer format, the rest is a file part send as text string that does not require the conversion.

Analyze how 5 connection limit works, pay attention how “free” member in the connections structure works, how it is affected by the last part marker in the datagram?

Multithreaded Servers

During the previous lab, we practiced writing servers using only one thread. Such architecture makes a lot of sense when we need to conserve resources and expect a relatively low load.

Often, however, our server must handle a very large number of requests. In such a situation, to achieve adequate performance on modern hardware, it is necessary to use multiple threads. A typical and natural architecture is one thread receiving messages and passing tasks to worker threads, which process them and send results back to clients. On the other hand, the atomicity of operations on datagrams also allows many threads to wait simultaneously for a message on a single socket. Of course, in such a situation there is often some additional state associated with it, so synchronization may still be necessary, for example using a mutex.

To write efficient multithreaded programs during the lab, it is worth rereading lab 4 tutorial (synchronization), in particular mutexes, semaphores, and condition variables. Review also the typical data structures used for this kind of task, such as a thread pool or a circular buffer. If you do not remember these topics well, look through the tutorials for Lab 3 and Lab 4, as well as the slides and lecture programs on synchronization.

Example Tasks

Complete the sample exercises. You will have more time and starter code during the lab session, but completing the tasks below on your own means you are well prepared.

- Task 1 ~100 ~~days~~ minutes
- Task 2 from Lab 7 ~120 minutes total, stages 4–5 concern Lab 8
- we do not have more specific tasks, but tasks from Lab 5 and Lab 7 are well suited for practicing the topic - simply rewrite the communication so that it uses UDP.

Source codes presented in this tutorial

- l8_common.h
- l8-1_client.c
- l8-1_server.c

Other references

- https://cs341.cs.illinois.edu/coursebook/Networking#layer-4-udp